

Παράλληλη Επεξεργασία

Παράλληλη Προεπεξεργασία Μεγάλου Όγκου Δεδομένων

Τμήμα Μηχανικών Η/Υ και Πληροφορικής, Πανεπιστήμιο Πατρών

Εαρινό Εξάμηνο 2025-2026

Ονοματεπώνυμο	Αριθμός Μητρώου	Email	Έτος
Μανταδάκης Σπυρίδων	1100613	up1100613@ac.upatras.gr	4ο
Ιωάννης Κώτσαλος	1100603	up1100603@ac.upatras.gr	4ο

Περιεχόμενα

1. Εισαγωγή.....	4
2. Περιγραφή προβλήματος και μορφή δεδομένων	4
3. Περιβάλλον ανάπτυξης και μετρήσεων.....	4
3.1 Development περιβάλλον.....	5
3.2 Benchmark περιβάλλον	5
4. Περιγραφή αρχείων	5
5. Κοινός αλγόριθμος δύο φάσεων.....	6
5.1 Φάση 1 - Υπολογισμός στατιστικών	6
5.2 Φάση 2 - Μετασχηματισμός και εγγραφή.....	7
6. Σειριακή υλοποίηση αναφοράς.....	7
7. SIMD υλοποίηση	7
7.1 Vectorized τμήματα	7
7.2 Βελτιστοποιήσεις μνήμης και I/O	8
8. OpenMP υλοποίηση.....	8
8.1 Κατανομή εργασίας και αποφυγή race conditions.....	8
8.2 Μετρήσεις OpenMP	8
9. MPI υλοποίηση	9
9.1 Row partitioning.....	9
9.2 Υπολογισμός καθολικών στατιστικών.....	9
9.3 Scaling και εγγραφή	9
10. CUDA υλοποίηση	10
10.1 Kernels.....	10
10.2 Host orchestration και overlap	10
11. Παραγωγή δεδομένων.....	10
12. Έλεγχος ορθότητας	11
13. Πειραματική αξιολόγηση	12
13.1 Μεθοδολογία.....	12
13.2 OpenMP scaling	13
13.3 MPI scaling	14
13.4 CUDA timing breakdown.....	15
14. Συμπεράσματα.....	15
Παράρτημα Α: Ενδεικτικές εντολές εκτέλεσης.....	17
Build μέσω Makefile.....	17

Παραγωγή δεδομένων.....	17
Εκτέλεση εκδόσεων	17
Έλεγχος ορθότητας	17
Άμεση μεταγλώττιση χωρίς Makefile	17

1. Εισαγωγή

Η εργασία αφορά την αποδοτική προεπεξεργασία μεγάλων αριθμητικών δεδομένων πριν από την εφαρμογή αλγορίθμων μηχανικής μάθησης ή ανάλυσης δεδομένων. Το πρόγραμμα διαβάζει έναν πίνακα πραγματικών αριθμών από raw binary αρχείο, υπολογίζει στατιστικά ανά στήλη και παράγει νέο αρχείο με κανονικοποιημένες τιμές.

Η επεξεργασία μεγάλων αρχείων δεν είναι μόνο υπολογιστικό πρόβλημα. Όταν τα δεδομένα έχουν μέγεθος πολλών GB, ο χρόνος εκτέλεσης επηρεάζεται έντονα από το I/O, το memory bandwidth, τη διαθέσιμη RAM, τη διάταξη των δεδομένων και τον τρόπο αξιοποίησης CPU/GPU. Για τον λόγο αυτό όλες οι υλοποιήσεις ακολουθούν block-based, out-of-core λογική, ώστε να μη θεωρείται ότι ολόκληρος ο πίνακας χωράει στη μνήμη.

Υλοποιήθηκαν πέντε εκδόσεις: σειριακή έκδοση αναφοράς, SIMD με AVX2 intrinsics, OpenMP, MPI και CUDA. Επιπλέον υπάρχουν script παραγωγής δεδομένων, script ελέγχου ορθότητας και Makefile που ενοποιεί τη μεταγλώττιση και τις εκτελέσεις.

2. Περιγραφή προβλήματος και μορφή δεδομένων

Δίνεται πίνακας X με N γραμμές και D στήλες, αποθηκευμένος σε raw binary μορφή χωρίς header και χωρίς metadata. Τα δεδομένα είναι row-major, δηλαδή οι τιμές κάθε γραμμής αποθηκεύονται διαδοχικά. Οι διαστάσεις N και D δίνονται από τη γραμμή εντολών.

Παράμετρος	Περιγραφή
input.bin	Αρχικό raw binary αρχείο εισόδου
output.bin	Raw binary αρχείο εξόδου μετά το scaling
N	Αριθμός γραμμών/δειγμάτων
D	Αριθμός στηλών/χαρακτηριστικών
mode	standard για StandardScaler ή minmax για MinMaxScaler
block_rows	Προαιρετικός αριθμός γραμμών ανά block, default 256000

Στον κώδικα οι δύο μετασχηματισμοί ενοποιούνται μέσω πινάκων shift και scale ανά στήλη. Για StandardScaler ισχύει $\text{shift} = \text{mean}$ και $\text{scale} = 1/\text{std_dev}$, ενώ για MinMaxScaler ισχύει $\text{shift} = \text{min}$ και $\text{scale} = 1/(\text{max} - \text{min})$. Αν παρουσιαστεί μηδενική τυπική απόκλιση ή μηδενικό εύρος, το scale τίθεται σε 0 ώστε να αποφευχθεί διαίρεση με το μηδέν.

3. Περιβάλλον ανάπτυξης και μετρήσεων

Η ανάπτυξη και ο αρχικός έλεγχος έγιναν σε προσωπικό Windows PC. Οι συγκριτικές μετρήσεις απόδοσης πρέπει να γίνουν στο ίδιο τελικό περιβάλλον, δηλαδή στο μηχάνημα krylon100 του HPC lab, ώστε τα speedsups να είναι συγκρίσιμα.

3.1 Development περιβάλλον

Πεδίο	Τιμή
Operating System	Windows 10, Microsoft Windows NT 10.0.19045.0
CPU	Intel(R) Core(TM) i5-9400F CPU @ 2.90GHz
Cores / logical processors	6 logical processors όπως αναφέρθηκαν από το τοπικό περιβάλλον
RAM	Περίπου 16 GB
GPU	Δεν χρησιμοποιήθηκε για τις τελικές CUDA μετρήσεις
Compiler διαθέσιμος τοπικά	MinGW g++ 6.3.0
Ρόλος	Ανάπτυξη, επιμέλεια κώδικα και μικροί έλεγχοι.

3.2 Benchmark περιβάλλον

Πεδίο	Τιμή
Μηχάνημα	krylov100, προσβάσιμο ως scgroup3.ceid.upatras.gr
Περιγραφή	Server με 24 Intel Xeon cores, 128 GB RAM και μία NVIDIA Tesla V100 GPU
CPU	24 Intel Xeon cores
RAM	128 GB
GPU	NVIDIA Tesla V100, 5120 CUDA cores, 32 GB graphics memory
Compiler	g++ για Serial/SIMD/OpenMP, mpicxx για MPI, nvcc για CUDA
CUDA στόχος	Tesla V100, compute capability sm_70. Στο Makefile χρησιμοποιείται /usr/local/cuda-12.2/bin/nvcc με -arch=sm_70.
MPI	Η MPI υλοποίηση μεταγλωττίζεται με mpicxx και εκτελείται με mpirun.
Filesystem	Το ίδιο filesystem πρέπει να χρησιμοποιηθεί για όλες τις εκδόσεις των benchmarks, επειδή η εφαρμογή είναι έντονα I/O-bound.

4. Περιγραφή αρχείων

Αρχείο	Ρόλος
--------	-------

generate_data_chunks.py	Παράγει συνθετικά δεδομένα με sklearn.datasets.make_regression σε chunks, ώστε να αποφεύγεται η υπερφόρτωση μνήμης κατά τη δημιουργία μεγάλων datasets. Χρησιμοποιήθηκε για το development.
Scaler_Serial.cpp	Σειριακή έκδοση αναφοράς με block-based ανάγνωση, υπολογισμό στατιστικών ανά στήλη και εφαρμογή StandardScaler/MinMaxScaler.
Scaler_SIMD.cpp	SIMD έκδοση με AVX2 intrinsics, FMA, aligned buffers, scalar fallback και double-buffered ανάγνωση με std::async.
Scaler_OpenMP.cpp	Πολυνηματική έκδοση OpenMP με thread-local accumulators στη Φάση 1 και parallel for στη Φάση 2.
Scaler_MPI.cpp	Κατανεμημένη έκδοση MPI με row-wise partitioning, MPI-IO, MPI_Allreduce για τα καθολικά στατιστικά και ανεξάρτητη εγγραφή των partitions.
Scaler_CUDA.cu	CUDA έκδοση για GPU με pinned host memory, CUDA streams, double buffering, kernels για statistics και scaling, και αναλυτικές μετρήσεις H2D/kernel/D2H.
Verifier.py	Streaming έλεγχος ορθότητας με NumPy. Υπολογίζει max absolute error, mean absolute error και μετα-scaling στατιστικά.
Makefile	Κεντρικά targets για build, παραγωγή δεδομένων, εκτέλεση όλων των εκδόσεων, verification και clean.

5. Κοινός αλγόριθμος δύο φάσεων

Όλες οι υλοποιήσεις ακολουθούν την ίδια αλγοριθμική δομή δύο φάσεων. Αυτό είναι απαραίτητο επειδή το scaling κάθε στοιχείου εξαρτάται από καθολικά στατιστικά της αντίστοιχης στήλης, τα οποία είναι γνωστά μόνο αφού διαβαστεί ολόκληρο το input.

5.1 Φάση 1 - Υπολογισμός στατιστικών

Το input διαβάζεται σε blocks B γραμμών. Για κάθε block ενημερώνονται ανά στήλη τα sum , sum_sq , min και max . Μετά την ολοκλήρωση της σάρωσης υπολογίζονται mean , variance και std_dev . Η διακύμανση υπολογίζεται ως $E[x^2] - \text{mean}^2$ και μικρές αρνητικές τιμές που μπορεί να προκύψουν από στρογγυλοποίηση διορθώνονται σε 0.

5.2 Φάση 2 - Μετασχηματισμός και εγγραφή

Το input διαβάζεται δεύτερη φορά σε blocks. Με βάση το mode δημιουργούνται οι πίνακες shift και scale. Κάθε στοιχείο μετασχηματίζεται in-place με τη σχέση $(x - \text{shift}[j]) * \text{scale}[j]$ και το block γράφεται στο output αρχείο με την ίδια row-major σειρά.

Η συνολική κίνηση δεδομένων είναι περίπου $3 * \text{file_size}$: μία πλήρης ανάγνωση στη Φάση 1, μία δεύτερη ανάγνωση στη Φάση 2 και μία πλήρης εγγραφή του output. Για αυτό η αναφορά χρησιμοποιεί $\text{throughput} = 3 * \text{file_size} / \text{total_time}$.

6. Σειριακή υλοποίηση αναφοράς

Η σειριακή έκδοση βρίσκεται στο Scaler_Serial.cpp και λειτουργεί ως baseline για correctness και speedup. Χρησιμοποιεί C++17, std::vector για τα blocks και απλά nested loops πάνω σε γραμμές και στήλες.

- Υποστηρίζει raw binary αρχεία double χωρίς header.
- Δέχεται N, D, mode και προαιρετικό block_rows από τη γραμμή εντολών.
- Υπολογίζει sum, sum_sq, min, max, mean, variance και std_dev ανά στήλη.
- Εφαρμόζει StandardScaler και MinMaxScaler μέσω κοινών shift/scale πινάκων.
- Μετρά ξεχωριστά wall time και compute time για κάθε φάση, ώστε να εκτιμάται έμμεσα το κόστος I/O.

Μεταγλώττιση:

```
g++ -std=c++17 -O2 -Wall -o scaler_serial Scaler_Serial.cpp
```

7. SIMD υλοποίηση

Η SIMD έκδοση βρίσκεται στο Scaler_SIMD.cpp. Χρησιμοποιεί explicit AVX2 intrinsics όταν ο compiler ορίζει __AVX2__, ενώ διαθέτει scalar fallback για περιβάλλοντα χωρίς AVX2. Η vectorization γίνεται κατά μήκος των στηλών κάθε γραμμής, εκμεταλλευόμενη ότι τα στοιχεία μιας γραμμής είναι συνεχόμενα στη μνήμη.

7.1 Vectorized τμήματα

- Στη Φάση 1 κάθε AVX2 register επεξεργάζεται 4 double τιμές ταυτόχρονα.
- Τα sum ενημερώνονται με _mm256_add_pd.
- Τα sum_sq ενημερώνονται με _mm256_fmadd_pd, ώστε ο πολλαπλασιασμός $x*x$ και η πρόσθεση να γίνονται με FMA.
- Τα min και max ενημερώνονται με _mm256_min_pd και _mm256_max_pd.
- Στη Φάση 2 φορτώνονται ανά τετράδα οι τιμές row[j], shift[j], scale[j] και υπολογίζεται το $(x - \text{shift}) * \text{scale}$ με AVX2 πράξεις.

7.2 Βελτιστοποιήσεις μνήμης και I/O

- Οι βοηθητικοί πίνακες και τα blocks δεσμεύονται με 32-byte alignment μέσω `_mm_malloc` ή `posix_memalign`.
- Όταν D είναι πολλαπλάσιο του 4, οι row starts παραμένουν aligned και χρησιμοποιούνται aligned loads/stores. Διαφορετικά χρησιμοποιούνται unaligned loads/stores και scalar tail.
- Η ανάγνωση γίνεται με double buffering: ενώ το τρέχον block επεξεργάζεται, το επόμενο διαβάζεται με `std::async`.
- Χρησιμοποιούνται μεγάλα stdio buffers 8 MB για μείωση system call overhead.
- Σε Linux καλείται `posix_fadvise` με `POSIX_FADV_SEQUENTIAL`, ώστε το λειτουργικό να κάνει πιο επιθετικό read-ahead.

Μεταγλώττιση:

```
g++ -std=c++17 -O3 -Wall -mavx2 -mfma -march=native -pthread -o scaler_simd Scaler_SIMD.cpp
```

8. OpenMP υλοποίηση

Η OpenMP έκδοση βρίσκεται στο `Scaler_OpenMP.cpp` και στοχεύει στην αξιοποίηση των πολλών πυρήνων του `krylon100`. Η δομή I/O παραμένει block-based όπως στη σειριακή έκδοση, ενώ η επεξεργασία μέσα σε κάθε block εκτελείται παράλληλα.

8.1 Κατανομή εργασίας και αποφυγή race conditions

- Στη Φάση 1 δημιουργείται πίνακας `local_stats` με μέγεθος `max_threads * D`.
- Κάθε thread αρχικοποιεί και ενημερώνει μόνο το δικό του τμήμα `local_stats`, άρα δεν υπάρχουν ταυτόχρονες εγγραφές στα ίδια per-column accumulators.
- Η κατανομή γραμμών γίνεται με `#pragma omp for schedule(static)`. Κάθε thread επεξεργάζεται ανεξάρτητες γραμμές του block.
- Μετά το parallel region γίνεται σειριακή συγχώνευση των thread-local accumulators στα συνολικά stats.
- Στη Φάση 2 χρησιμοποιείται `#pragma omp parallel for schedule(static)` πάνω στις γραμμές. Δεν υπάρχουν races επειδή κάθε thread γράφει διαφορετικά στοιχεία του block.

8.2 Μετρήσεις OpenMP

Ο αριθμός threads ελέγχεται από την `OMP_NUM_THREADS` μεταβλητή, την οποία το Makefile περνά μέσω `OMP_THREADS`.

Μεταγλώττιση:

```
g++ -std=c++17 -O3 -Wall -fopenmp -o scaler_openmp Scaler_OpenMP.cpp
```


9. MPI υλοποίηση

Η MPI έκδοση βρίσκεται στο `Scaler_MPI.cpp` και κατανέμει τις γραμμές του πίνακα σε MPI processes. Αν και το `krylon100` είναι ένας κόμβος, η υλοποίηση ακολουθεί μοντέλο κατανεμημένης μνήμης και χρησιμοποιεί MPI-IO για άμεση πρόσβαση κάθε process στο δικό του τμήμα του αρχείου.

9.1 Row partitioning

Για P processes, κάθε rank παίρνει συνεχόμενο διάστημα γραμμών. Ο υπολογισμός γίνεται με $\text{base} = N / P$ και $\text{remainder} = N \% P$. Τα πρώτα remainder ranks παίρνουν μία επιπλέον γραμμή, άρα η ανισορροπία είναι το πολύ μία γραμμή.

9.2 Υπολογισμός καθολικών στατιστικών

- Κάθε process διαβάζει το partition του με `MPI_File_read_at`, χρησιμοποιώντας `byte offset = row_start * D * sizeof(double)`.
- Κάθε process υπολογίζει τοπικά `sum`, `sum_sq`, `min` και `max` ανά στήλη.
- Τα καθολικά στατιστικά συγχωνεύονται με τέσσερα `MPI_Allreduce`: `SUM` για `sum`, `SUM` για `sum_sq`, `MIN` για `min` και `MAX` για `max`.
- Όλα τα ranks υπολογίζουν το ίδιο `mean`, `variance` και `std_dev` μετά το `Allreduce`.
- Το `communication cost` μετριέται ξεχωριστά ως χρόνος των `Allreduce` calls.

9.3 Scaling και εγγραφή

- Κάθε process επαναδιαβάζει το partition του, εφαρμόζει το `shift/scale in-place` και γράφει στο output με `MPI_File_write_at` στο ίδιο `byte offset`.
- Το output αρχείο δημιουργείται με `MPI_MODE_CREATE | MPI_MODE_WRONLY` και γίνεται `MPI_File_set_size` στο τελικό μέγεθος ώστε να μη μείνουν `stale bytes` από παλαιότερες εκτελέσεις.
- Οι χρόνοι `wall time` μειώνονται με `MPI_Allreduce MAX`, ώστε να αναφέρεται ο χρόνος του πιο αργού process, δηλαδή ο πραγματικός `parallel wall-clock` χρόνος.
- Υπάρχει `guard` ώστε `block_rows * D` να μην ξεπερνά το `INT_MAX`, επειδή τα MPI element counts είναι `int`.

Μεταγλώττιση και εκτέλεση:

```
mpicxx -std=c++17 -O3 -Wall -o scaler_mpi Scaler_MPI.cpp
mpirun -np 8 ./scaler_mpi input.bin output.bin N D standard 256000
```

10. CUDA υλοποίηση

Η CUDA έκδοση βρίσκεται στο `Scaler_CUDA.cu` και αξιοποιεί την NVIDIA Tesla V100 του krylon100. Η υλοποίηση είναι out-of-core: κάθε φορά μεταφέρεται και επεξεργάζεται ένα block, άρα μπορεί να δουλέψει και για datasets μεγαλύτερα από τη μνήμη της GPU.

10.1 Kernels

- Ο `phase1_kernel` υπολογίζει partial statistics στη GPU. Κάθε CUDA block χρησιμοποιεί shared memory για arrays `sum`, `sum_sq`, `min` και `max` μήκους `D`.
- Οι ενημερώσεις `sum` και `sum_sq` χρησιμοποιούν `atomicAdd` σε double precision.
- Για `min` και `max` υλοποιούνται custom `atomicMinDouble` και `atomicMaxDouble` με `atomicCAS`, επειδή απαιτείται double precision.
- Μετά τη shared-memory συγκέντρωση, τα αποτελέσματα κάθε CUDA block συγχωνεύονται σε global device arrays.
- Ο `phase2_kernel` αντιστοιχεί ένα CUDA thread σε ένα στοιχείο του block και εφαρμόζει `data[idx] = (data[idx] - shift[col]) * scale[col]`.

10.2 Host orchestration και overlap

- Χρησιμοποιούνται δύο pinned host buffers με `cudaMallocHost` και δύο device buffers με `cudaMalloc`.
- Η εκτέλεση γίνεται με δύο CUDA streams, ώστε H2D μεταφορές, kernels και D2H μεταφορές να μπορούν να επικαλύπτονται με disk I/O.
- Στη Φάση 1 μετρώνται ξεχωριστά H2D και kernel time. Στη Φάση 2 μετρώνται H2D, kernel και D2H time.
- Τα `shift` και `scale` αντιγράφονται μία φορά στη GPU πριν από τη Φάση 2.
- Ο κώδικας ελέγχει ότι $4 * D * \text{sizeof}(\text{double})$ χωράει στο `sharedMemPerBlock` της συσκευής. Για `D = 128` απαιτούνται 4096 bytes, άρα η V100 το υποστηρίζει άνετα.

Μεταγλώττιση στο krylon100 για Tesla V100:

```
/usr/local/cuda-12.2/bin/nvcc -std=c++17 -O3 -arch=sm_70 -o scaler_cuda Scaler_CUDA.cu
```

11. Παραγωγή δεδομένων

Το `generate_data_chunks.py` παράγει συνθετικά δεδομένα με τη συνάρτηση `make_regression` της `scikit-learn`. Η παραγωγή γίνεται chunk-by-chunk, ώστε να μπορούν να δημιουργηθούν μεγάλα raw binary αρχεία χωρίς να απαιτείται να χωρά ολόκληρος ο πίνακας στη μνήμη της Python.

- Οι βασικές παράμετροι είναι `--samples N`, `--features D`, `--output`, `--dtype`, `--seed`, `--noise` και `--chunk-size`.

12. Έλεγχος ορθότητας

Ο έλεγχος ορθότητας γίνεται με το Verifier.py. Το script διαβάζει το input και το output σε blocks, υπολογίζει reference αποτέλεσμα με NumPy και συγκρίνει το output κάθε υλοποίησης με το reference.

- Μετρώνται max absolute error και mean absolute error.
- Για StandardScaler ελέγχεται επιπλέον ότι οι post-scale μέσες τιμές είναι κοντά στο 0 και οι τυπικές αποκλίσεις κοντά στο 1.
- Για MinMaxScaler ελέγχεται ότι τα post-scale ελάχιστα είναι κοντά στο 0 και τα μέγιστα κοντά στο 1.
- Το default tolerance που αναφέρει το script είναι $1e-9$ για max absolute error.

Έκδοση	Dataset	Mode	Max abs error	Mean abs error	Post-scale έλεγχος	Αποτέλεσμα
Serial	Medium (5M×64)	standard	4.813927e-13	2.330138e-14	1.483293e-16, 1.874056e-13	PASS
Serial	Large (10M×128)	standard	1.008971e-12	3.182876e-14	1.483293e-16, 1.874056e-13	PASS
Serial	Medium (5M×64)	minmax	0.000000e+00	0.000000e+00	0.000000e+00, 1.110223e-16	PASS
Serial	Large (10M×128)	minmax	0.000000e+00	0.000000e+00	0.000000e+00, 1.110223e-16	PASS
SIMD	Medium (5M×64)	standard	4.805045e-13	2.330415e-14	9.840448e-17, 9.237056e-14	PASS
SIMD	Large (10M×128)	standard	1.007194e-12	3.182738e-14	1.483293e-16, 1.871836e-13	PASS
SIMD	Medium (5M×64)	minmax	0.000000e+00	0.000000e+00	0.000000e+00, 1.110223e-16	PASS
SIMD	Large (10M×128)	minmax	0.000000e+00	0.000000e+00	0.000000e+00, 1.110223e-16	PASS
OpenMP	Medium (5M×64)	standard	1.776357e-15	1.660716e-17	4.201297e-17, 5.662137e-15	PASS
OpenMP	Large (10M×128)	standard	1.687539e-14	6.759612e-16	3.801954e-17, 3.330669e-15	PASS
OpenMP	Medium (5M×64)	minmax	0.000000e+00	0.000000e+00	0.000000e+00, 1.110223e-16	PASS
OpenMP	Large (10M×128)	minmax	0.000000e+00	0.000000e+00	0.000000e+00, 1.110223e-16	PASS
MPI	Medium (5M×64)	standard	2.398082e-14	1.050088e-15	4.476988e-17, 4.884981e-15	PASS
MPI	Large (10M×128)	standard	2.309264e-13	6.102535e-15	6.510845e-17, 3.996803e-14	PASS
MPI	Medium (5M×64)	minmax	0.000000e+00	0.000000e+00	0.000000e+00, 1.110223e-16	PASS
MPI	Large (10M×128)	minmax	0.000000e+00	0.000000e+00	0.000000e+00, 1.110223e-16	PASS
CUDA	Medium (5M×64)	standard	3.197442e-14	1.428340e-15	3.440448e-17, 5.551115e-15	PASS
CUDA	Large (10M×128)	standard	5.595524e-14	2.149696e-15	3.372733e-17, 9.769963e-15	PASS

Έκδοση	Dataset	Mode	Max abs error	Mean abs error	Post-scale έλεγχος	Αποτέλεσμα
CUDA	Medium (5M×64)	minmax	0.000000e+00	0.000000e+00	0.000000e+00, 1.110223e-16	PASS
CUDA	Large (10M×128)	minmax	0.000000e+00	0.000000e+00	0.000000e+00, 1.110223e-16	PASS

13. Πειραματική αξιολόγηση

13.1 Μεθοδολογία

Καλύπτονται τα μεγέθη dataset (Medium, Large), και οι δύο τρόποι κανονικοποίησης, για κάθε υλοποίηση. $\text{Speedup} = T_{\text{serial}} / T_{\text{version}}$. $\text{Throughput} = 3 \times \text{file_size} / \text{total_time}$ (GB/s).

Καταγράφονται Phase 1 wall time, Phase 2 wall time, total time, compute-only time όπου υπάρχει, throughput και speedup.

Για OpenMP μεταβάλλεται το OMP_NUM_THREADS. Για MPI μεταβάλλεται το -np. Για CUDA χρησιμοποιείται η V100.

Έκδοση	Dataset	Mode	Phase 1 Compute, I/O (s)	Phase 2 Compute, I/O (s)	Total (s)	Throughput (GB/s)	Speedup
Serial	Medium (5M×64, ~2.56 GB)	standard	0.794, 9.161	0.285, 2.819	13.060	0.55	1.00 (baseline)
SIMD	Medium (5M×64, ~2.56 GB)	standard	0.341, 0.987	0.306, 2.485	4.120	1.74	3.17x
Serial	Medium (5M×64, ~2.56 GB)	minmax	0.999, 7.460	0.323, 3.808	12.590	0.57	1.00 (baseline)
SIMD	Medium (5M×64, ~2.56 GB)	minmax	0.331, 0.684	0.366, 2.513	3.893	1.84	3.23x
Serial	Large (10M×128, ~10.24 GB)	standard	2.942, 9.046	1.383, 91.206	104.578	0.27	1.00 (baseline)
SIMD	Large (10M×128, ~10.24 GB)	standard	1.205, 2.266	1.046, 9.445	13.962	2.05	7.49x
Serial	Large (10M×128, ~10.24 GB)	minmax	2.903, 2.555	1.231, 81.425	88.115	0.32	1.00 (baseline)
SIMD	Large (10M×128, ~10.24 GB)	minmax	1.269, 1.827	1.006, 9.135	13.236	2.16	6.66x

13.2 OpenMP scaling

Threads: 1, 2, 4, 8, 16, 24 — για Medium (5M×64) και Large (10M×128), και για standard και minmax. Speedup = $T_{\text{serial}} / T_{\text{OpenMP}(n)}$. Efficiency = $(\text{speedup} / n) \times 100\%$.

Dataset	Mode	Threads	Phase 1 Compute, I/O (s)	Phase 2 Compute, I/O (s)	Total time (s)	Throughput (GB/s)	Speedup	Efficiency
Medium (5M×64)	standard	1	0.737, 0.683	0.260, 3.039	4.719	1.52	1.00	100%
Medium (5M×64)	standard	2	0.582, 0.604	0.203, 2.975	4.364	1.64	1.08x	54.07%
Medium (5M×64)	standard	4	0.491, 0.663	0.182, 3.116	4.453	1.61	1.06x	26.49%
Medium (5M×64)	standard	8	0.311, 0.790	0.115, 3.046	4.262	1.68	1.11x	13.84%
Medium (5M×64)	standard	16	0.237, 0.715	0.146, 3.441	4.539	1.58	1.04x	6.50%
Medium (5M×64)	standard	24	0.191, 0.737	0.104, 3.317	4.349	1.64	1.09x	4.52%
Medium (5M×64)	minmax	1	0.749, 0.802	0.247, 2.875	4.672	1.53	1.00	100%
Medium (5M×64)	minmax	2	0.600, 0.690	0.415, 3.143	4.848	1.48	0.96x	48.18%
Medium (5M×64)	minmax	4	0.490, 0.706	0.170, 3.405	4.771	1.50	0.98x	24.48%
Medium (5M×64)	minmax	8	0.308, 0.625	0.107, 2.880	3.921	1.82	1.19x	14.89%
Medium (5M×64)	minmax	16	0.201, 0.637	0.089, 3.068	3.995	1.79	1.17x	7.31%
Medium (5M×64)	minmax	24	0.151, 0.676	0.106, 3.063	3.996	1.79	1.17x	4.87%
Large (10M×128)	standard	1	2.807, 2.549	0.982, 11.655	17.993	1.59	1.00	100%
Large (10M×128)	standard	2	2.491, 2.986	1.049, 12.212	18.737	1.53	0.96x	48.01%
Large (10M×128)	standard	4	1.674, 2.768	0.738, 12.181	17.360	1.65	1.04x	25.91%
Large (10M×128)	standard	8	1.030, 2.867	0.484, 13.102	17.483	1.64	1.03x	12.86%
Large (10M×128)	standard	16	0.726, 3.215	0.421, 12.788	17.150	1.67	1.05x	6.56%
Large (10M×128)	standard	24	0.566, 2.803	0.495, 13.306	17.169	1.67	1.05x	4.37%
Large (10M×128)	minmax	1	2.818, 2.258	1.015, 11.530	17.621	1.62	1.00	100%
Large (10M×128)	minmax	2	1.884, 2.301	0.900, 11.932	17.017	1.68	1.04x	51.77%
Large (10M×128)	minmax	4	1.215, 2.286	0.590, 11.577	15.667	1.83	1.12x	28.12%
Large (10M×128)	minmax	8	0.970, 2.434	0.486, 11.850	15.740	1.82	1.12x	13.99%
Large (10M×128)	minmax	16	0.682, 2.518	0.501, 11.982	15.683	1.82	1.12x	7.02%
Large (10M×128)	minmax	24	0.615, 2.883	0.571, 12.614	16.684	1.71	1.06x	4.40%

13.3 MPI scaling

Processes: 1, 2, 4, 8, 16, 24 — για Medium (5M×64) και Large (10M×128), και για standard και minmax.

Dataset	Mode	Processes	Phase 1 Compute, I/O (s)	Phase 2 Compute, I/O (s)	Total time (s)	Throughput (GB/s)	Speedup
Medium (5M×64)	standard	1	0.646, 0.658	0.254, 2.459	4.017	1.78	1.00
Medium (5M×64)	standard	2	0.321, 0.339	0.133, 1.802	2.595	2.76	1.55x
Medium (5M×64)	standard	4	0.153, 0.193	0.116, 1.838	2.286	2.30	1.76x
Medium (5M×64)	standard	8	0.072, 0.136	0.091, 1.987	2.286	3.13	1.76x
Medium (5M×64)	standard	16	0.042, 0.108	0.044, 2.024	2.218	3.22	1.81x
Medium (5M×64)	standard	24	0.032, 0.144	0.043, 2.108	2.327	3.07	1.73x
Medium (5M×64)	minmax	1	0.642, 0.694	0.298, 2.475	4.109	1.74	1.00
Medium (5M×64)	minmax	2	0.317, 0.339	0.131, 0.976	1.763	4.06	2.33x
Medium (5M×64)	minmax	4	0.154, 0.186	0.066, 1.155	1.562	4.58	2.63x
Medium (5M×64)	minmax	8	0.081, 0.110	0.039, 1.253	1.483	4.82	2.77x
Medium (5M×64)	minmax	16	0.044, 0.098	0.042, 1.314	1.499	4.77	2.74x
Medium (5M×64)	minmax	24	0.030, 0.140	0.039, 1.347	1.556	4.60	2.64x
Large (10M×128)	standard	1	2.689, 2.649	1.078, 10.132	16.549	1.73	1.00
Large (10M×128)	standard	2	1.461, 1.343	0.725, 3.621	7.150	4.00	2.31x
Large (10M×128)	standard	4	0.716, 0.699	0.293, 4.429	6.137	4.66	2.70x
Large (10M×128)	standard	8	0.305, 0.485	0.174, 4.556	5.520	5.18	3.00x
Large (10M×128)	standard	16	0.170, 0.363	0.157, 4.724	5.415	5.28	3.06x
Large (10M×128)	standard	24	0.107, 0.507	0.125, 5.202	5.942	4.81	2.79x
Large (10M×128)	minmax	1	2.517, 2.666	1.028, 10.150	16.361	1.75	1.00

Dataset	Mode	Processes	Phase 1 Compute, I/O (s)	Phase 2 Compute, I/O (s)	Total time (s)	Throughput (GB/s)	Speedup
Large (10M×128)	minmax	2	1.388, 1.499	0.768, 3.643	7.298	3.92	2.24x
Large (10M×128)	minmax	4	0.703, 0.706	0.482, 4.240	6.132	4.67	2.67x
Large (10M×128)	minmax	8	0.316, 0.465	0.261, 4.596	5.637	5.08	2.90x
Large (10M×128)	minmax	16	0.177, 0.366	0.168, 4.736	5.446	5.25	3.00x
Large (10M×128)	minmax	24	0.205, 0.401	0.073, 4.788	5.467	5.23	2.99x

13.4 CUDA timing breakdown

Αναλυτική κατανομή χρόνου CUDA για Medium (5M×64), Large (10M×128) και τους δύο τρόπους κανονικοποίησης. Ph1: Φάση στατιστικών, Ph2: Φάση μετασχηματισμού. H2D = Host→Device, D2H = Device→Host.

Dataset	Mode	Ph1 H2D (s)	Ph1 kernel (s)	Ph2 H2D (s)	Ph2 kernel (s)	Ph2 D2H (s)	Total wall (s)
Medium (5M×64)	standard	0.213	0.048	0.211	0.033	0.199	4.040
Medium (5M×64)	minmax	0.210	0.047	0.211	0.035	0.199	4.171
Large (10M×128)	standard	0.853	0.096	0.844	0.070	0.791	14.980
Large (10M×128)	minmax	0.850	0.099	0.839	0.078	0.791	15.180

14. Συμπεράσματα

Η εργασία αυτή υλοποίησε και αξιολόγησε πέντε εκδόσεις παράλληλης προεπεξεργασίας μεγάλων αριθμητικών δεδομένων: σειριακή, SIMD, OpenMP, MPI και CUDA. Τα πειραματικά αποτελέσματα ανέδειξαν ότι η εφαρμογή είναι έντονα I/O-bound, γεγονός που επηρεάζει καθοριστικά την αποτελεσματικότητα κάθε τεχνικής παραλληλοποίησης.

Σειριακή έκδοση. Η σειριακή υλοποίηση αναφοράς επιτυγχάνει throughput περίπου 0.55 GB/s για το Medium dataset και μόνο 0.27 GB/s για το Large dataset. Η σημαντική πτώση απόδοσης για μεγάλα αρχεία οφείλεται στο cold-cache I/O: το Large dataset δεν χωρά στη cache του λειτουργικού συστήματος, με αποτέλεσμα χρόνο Phase 2 I/O που φτάνει τα 91 δευτερόλεπτα. Αυτό αναδεικνύει ότι ο κύριος περιορισμός της εφαρμογής δεν είναι ο υπολογισμός, αλλά η ανάγνωση και εγγραφή στο αποθηκευτικό σύστημα.

SIMD. Η SIMD έκδοση επέτυχε τα πιο εντυπωσιακά speedups: 3.17–3.23x για το Medium και έως 7.49x για το Large dataset. Αυτή η βελτίωση δεν προέρχεται αποκλειστικά από τα AVX2 intrinsics, αλλά κυρίως από τις βελτιστοποιήσεις I/O που συνοδεύουν την υλοποίηση: double buffering με std::async, μεγάλα stdio buffers 8 MB και posix_fadvise για επιθετικό read-ahead. Αυτές οι τεχνικές επιτρέπουν την επικάλυψη υπολογισμού και I/O, μειώνοντας δραστικά το χρόνο αναμονής από το δίσκο. Το throughput φτάνει τα 2.16 GB/s για το Large dataset, τετραπλάσιο της σειριακής έκδοσης.

OpenMP. Η OpenMP έκδοση παρουσίασε περιορισμένη βελτίωση απόδοσης. Για το Medium dataset, το μέγιστο speedup ήταν μόλις 1.19x (8 threads, minmax), ενώ για το Large dataset η βελτίωση δεν ξεπέρασε το 1.12x ακόμη και με 24 threads. Η απόδοση παραμένει ουσιαστικά αμετάβλητη με την αύξηση των threads, επειδή ο χρόνος I/O κυριαρχεί πλήρως. Η επιτάχυνση του υπολογιστικού τμήματος που επιτυγχάνει το OpenMP δεν αντισταθμίζει τον χρόνο αναμονής από το δίσκο, ο οποίος παραμένει σειριακός. Η efficiency πέφτει απότομα, φτάνοντας κάτω από 5% για 24 threads, γεγονός που επιβεβαιώνει ότι η παραλληλοποίηση υπολογισμού δεν αρκεί για I/O-bound εφαρμογές.

MPI. Η MPI έκδοση επέδειξε ουσιαστικά καλύτερη κλιμάκωση σε σχέση με το OpenMP, επιτυγχάνοντας speedup έως 3.06x για το Large dataset (16 processes, standard). Ο λόγος είναι ότι το MPI-IO επιτρέπει σε κάθε process να διαβάζει και να γράφει ανεξάρτητα το δικό του partition του αρχείου παράλληλα, αξιοποιώντας έτσι το bandwidth του filesystem πολύ αποτελεσματικότερα από την OpenMP. Η απόδοση κορυφώνεται γύρω στα 16 processes και μειώνεται ελαφρά στα 24, γεγονός που υποδηλώνει contention στο filesystem όταν υπάρχουν πολλά παράλληλα I/O requests. Το κόστος των MPI_Allreduce για τη συγχώνευση των στατιστικών ήταν αμελητέο σε σχέση με το I/O.

CUDA. Η CUDA έκδοση επέτυχε συνολικούς χρόνους συγκρίσιμους με τη SIMD και την MPI. Ωστόσο, η ανάλυση του timing breakdown αποκαλύπτει μια σημαντική διαπίστωση: ο χρόνος εκτέλεσης των kernels στη GPU είναι εξαιρετικά μικρός σε σύγκριση με το συνολικό χρόνο. Για το Large dataset, τα kernels Phase 1 και Phase 2 εκτελούνται σε συνολικά 0.166 δευτερόλεπτα, ενώ το συνολικό wall time φτάνει τα 14.98 δευτερόλεπτα. Αυτό σημαίνει ότι πάνω από το 98% του χρόνου αναλώνεται σε disk I/O και μεταφορές H2D/D2H. Η GPU υπολογίζει με εξαιρετική ταχύτητα, αλλά η εφαρμογή δεν είναι compute-bound και επομένως η υπολογιστική ισχύς της V100 δεν αξιοποιείται πλήρως. Σε ένα περιβάλλον με γρηγορότερο αποθηκευτικό σύστημα (π.χ. NVMe SSD ή in-memory dataset), η CUDA θα παρουσίαζε σαφώς ανώτερη απόδοση.

Γενικά συμπεράσματα. Το κεντρικό συμπέρασμα της εργασίας είναι ότι για I/O-bound εφαρμογές, η αποτελεσματικότητα της παραλληλοποίησης εξαρτάται κυρίως από το αν και κατά πόσο παραλληλοποιείται και το I/O, όχι μόνο ο υπολογισμός. Η SIMD βελτιστοποίησε το I/O μέσω double buffering, η MPI το παραλληλοποίησε μέσω MPI-IO, ενώ το OpenMP και η CUDA αντιμετωπίζουν τον δίσκο ως bottleneck που δεν μπορούν να παρακάμψουν. Η επιλογή της καταλληλότερης τεχνικής παραλληλοποίησης πρέπει να βασίζεται στην ανάλυση του πραγματικού bottleneck της εφαρμογής.

Παράρτημα Α: Ενδεικτικές εντολές εκτέλεσης

Build μέσω Makefile

```
make build
make scaler_serial
make scaler_simd
make scaler_openmp
make scaler_mpi
make scaler_cuda
```

Παραγωγή δεδομένων

```
make gen-data (στατικές default τιμές)
make gen-data N=1000000 D=32 INPUT_DATA=data_1000000_32.bin DTYPE=float64 SEED=42
make gen-data N=5000000 D=64 INPUT_DATA=data_5000000_64.bin DTYPE=float64 SEED=42
make gen-data N=10000000 D=128 INPUT_DATA=data_10000000_128.bin DTYPE=float64 SEED=42
```

Εκτέλεση εκδόσεων

```
make run-serial (στατικές default τιμές)
make run-serial N=1000000 D=32 INPUT_DATA=data_1000000_32.bin
OUT_DATA=out_serial_standard.bin MODE=standard BLOCKS=256000

make run-simd (στατικές default τιμές)
make run-simd N=1000000 D=32 INPUT_DATA=data_1000000_32.bin OUT_DATA=out_simd_standard.bin
MODE=standard BLOCKS=256000

make run-openmp (στατικές default τιμές)
make run-openmp OMP_THREADS=16 N=1000000 D=32 INPUT_DATA=data_1000000_32.bin
OUT_DATA=out_openmp_standard.bin MODE=standard BLOCKS=256000

make run-mpi (στατικές default τιμές)
make run-mpi NP=8 N=1000000 D=32 INPUT_DATA=data_1000000_32.bin
OUT_DATA=out_mpi_standard.bin MODE=standard BLOCKS=256000

make run-cuda (στατικές default τιμές)
make run-cuda N=1000000 D=32 INPUT_DATA=data_1000000_32.bin OUT_DATA=out_cuda_standard.bin
MODE=standard BLOCKS=256000
```

Έλεγχος ορθότητας

```
make verify (στατικές default τιμές, χρήση τελευταίου output αρχείου)
make verify N=1000000 D=32 INPUT_DATA=data_1000000_32.bin OUT_DATA=out_serial_standard.bin
MODE=standard DTYPE=float64 BLOCKS=256000
make verify N=1000000 D=32 INPUT_DATA=data_1000000_32.bin OUT_DATA=out_cuda_standard.bin
MODE=standard DTYPE=float64 BLOCKS=256000
```

Άμεση μεταγλώττιση χωρίς Makefile

```
g++ -std=c++17 -O2 -Wall -o scaler_serial Scaler_Serial.cpp
g++ -std=c++17 -O3 -Wall -mavx2 -mfma -march=native -pthread -o scaler_simd Scaler_SIMD.cpp
g++ -std=c++17 -O3 -Wall -fopenmp -o scaler_openmp Scaler_OpenMP.cpp
mpicxx -std=c++17 -O3 -Wall -o scaler_mpi Scaler_MPI.cpp
/usr/local/cuda-12.2/bin/nvcc -std=c++17 -O3 -arch=sm_70 -o scaler_cuda Scaler_CUDA.cu
```